

deeplearning.ai

Recurrent Neural Networks

Why sequence models?



deeplearning.ai

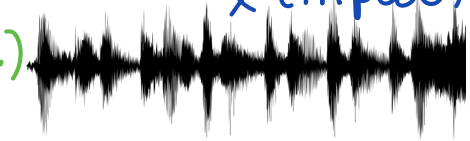
Discover recurrent neural networks, a type of model that performs extremely well on temporal data, and several of its variants, including LSTMs, GRUs and Bidirectional RNNs,
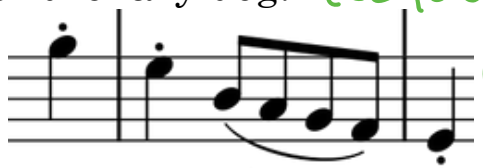
Learning Objectives

- Define notation for building sequence models
- Describe the architecture of a basic RNN
- Identify the main components of an LSTM
- Implement backpropagation through time for a basic RNN and an LSTM
- Give examples of several types of RNN
- Build a character-level text generation model using an RNN
- Store text data for processing using an RNN
- Sample novel sequences in an RNN
- Explain the vanishing/exploding gradient problem in RNNs
- Apply gradient clipping as a solution for exploding gradients
- Describe the architecture of a GRU
- Use a bidirectional RNN to take information from two points of a sequence
- Stack multiple RNNs on top of each other to create a deep RNN
- Use the flexible Functional API to create complex models
- Generate your own jazz music with deep learning
- Apply an LSTM to a music generation task

Examples of sequence data

x (input) — y (output)

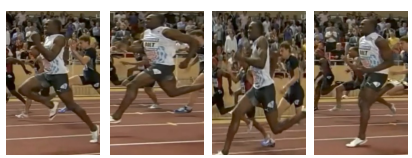
1. Speech recognition

(seq.) → "The quick brown fox jumped over the lazy dog." (sequence)

2. Music generation
genre / Ø / a few notes → 
(seq)

3. Sentiment classification
"There is nothing to like in this movie." (seq.) → ★☆☆☆☆

4. DNA sequence analysis
AGCCCCTGTGAGGAACTAG (sequence of DNA data) → AGCCCCTGTGAGGAACTAG
Detect protein sequence

5. Machine translation
Voulez-vous chanter avec moi? (seq) → Do you want to sing with me? (seq)

6. Video activity recognition

(seq. of frames) → Running

7. Name entity recognition
Yesterday, Harry Potter met Hermione Granger. (seq) → Yesterday, Harry Potter met Hermione Granger.
identify people

Andrew Ng

- All the above problems are supervised learning problems trained with input output pairs: (x, y)

- In some of the applications above, both the input x and the output Y are sequences.
  - → Sometimes x and y have different lengths: 1 and 5
  - → Sometimes x and y have the same length: Application 4 and 7
  - → Sometimes only x or y is a sequence: 2, 3 and 6

- The sequence models will be applicable to all of the above problems.



deeplearning.ai

Recurrent Neural Networks

Notation

Goal: Named-entity recognition: Given a sentence automatically tell where the peoples' names are in the sequence.

- Used by search engines to index the news by peoples' names.
- Named-entity recognition can be used to find peoples' names, companies names, times, locations, country names etc. in different types of text.

Motivating example: NLP: Natural Language Processing

x: Harry Potter and Hermione Granger invented a new spell.

$x^{<1>}$ $x^{<2>}$ $x^{<3>}$ ---- $x^{<t>}$ ---- $x^{<8>}$ $x^{<9>}$

$T_x = 9$

y: [1 1 0 1 1 0 0 0 0] 1 x 9

$y^{<1>}$ $y^{<2>}$ $y^{<3>}$ ---- $y^{<9>}$

$T_y = 9$

$x^{(i)<t>}$: t^{th} element in the sequence of training example i

$y^{(i)<t>}$: t^{th} element in the output sequence of training example i

$T_x^{(i)}$: input sequence length of training example i

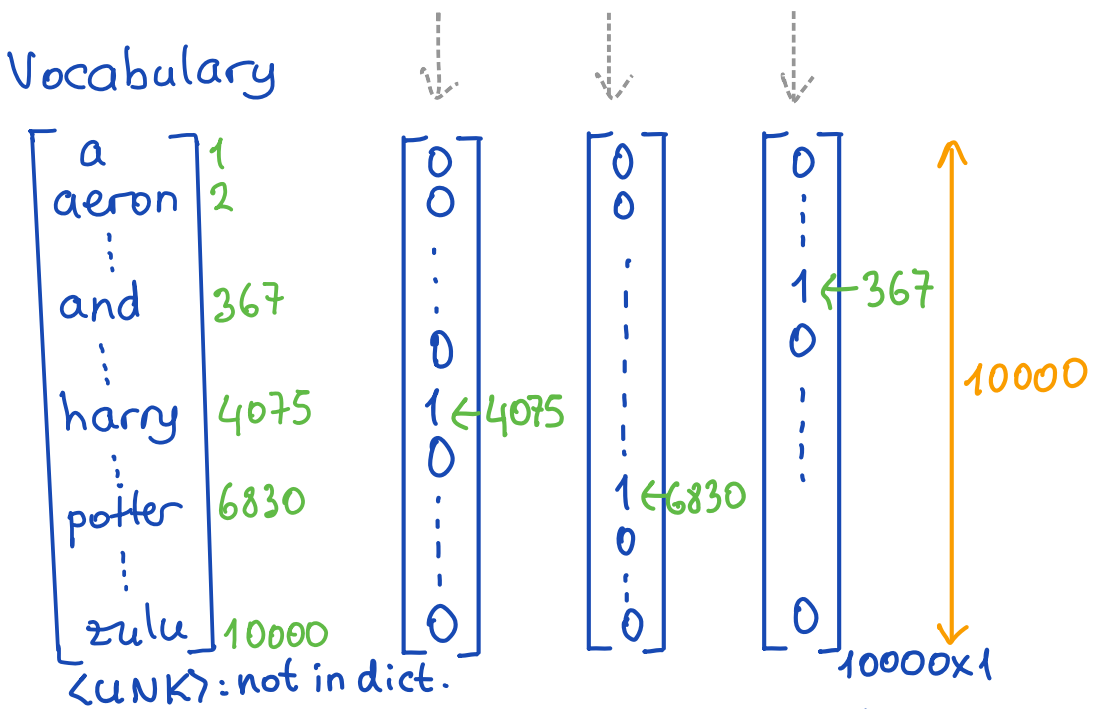
$T_y^{(i)}$: output sequence length of training example i

Andrew Ng

Representing words : one-hot encoding

x: Harry Potter and Hermione Granger invented a new spell.

$x^{<1>}$ $x^{<2>}$ $x^{<3>}$ ... $x^{<7>}$ $x^{<9>}$



- Note: Dictionary sizes are much larger in practice (30K, 50K, 1M words)
- Goal: Given the representation for X, learn a mapping using a sequence model to output y, using supervised learning.

Andrew Ng

Representing words

x: Harry Potter and Hermione Granger invented a new spell.

$x^{<1>}$ $x^{<2>}$ $x^{<3>}$ … $x^{<9>}$

And = 367
Invented = 4700
A = 1
New = 5976
Spell = 8376
Harry = 4075
Potter = 6830
Hermione = 4200
Gran… = 4000

Andrew Ng

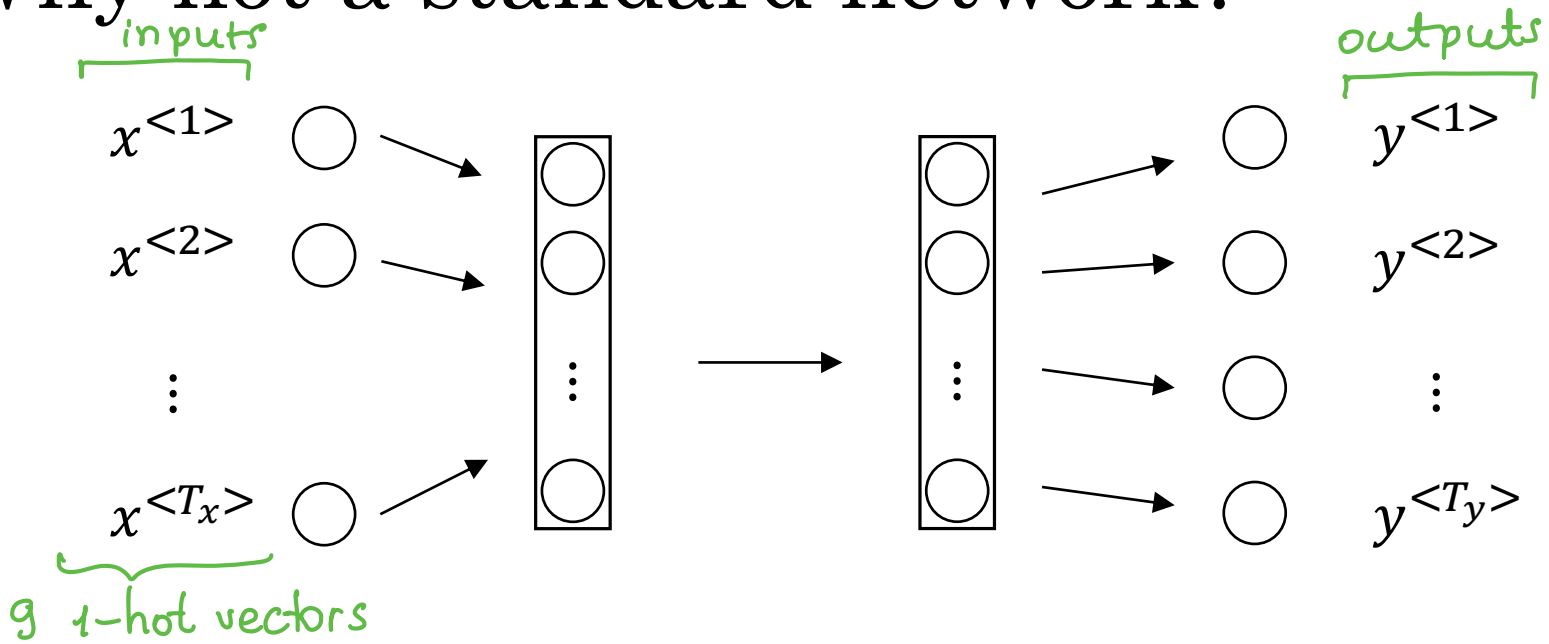


deeplearning.ai

Recurrent Neural Networks

Recurrent Neural Network Model

Why not a standard network?



Problems:

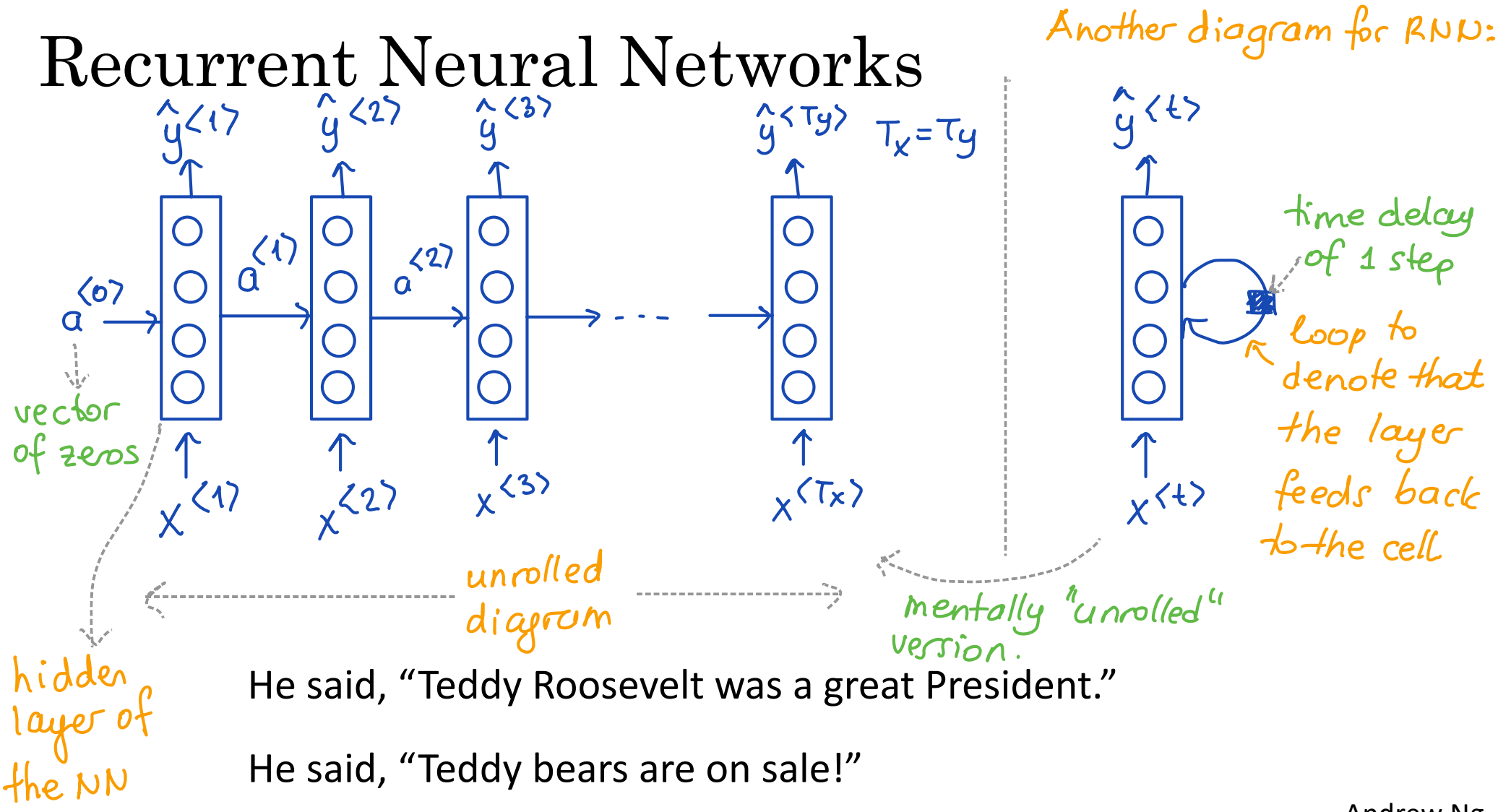
- Inputs, outputs can be different lengths in different examples.
- Doesn't share features learned across different positions of text.

Andrew Ng

- e.g. if the neural network has learned that the word "Harry" appearing in position one gives a sign that that's part of a person's name, then wouldn't it be nice if it automatically figures out that Harry appearing in some other position $x^{<t>}$ also means that that might be a person's name.
- This may be similar to what you saw in CNNs where you want things learned for one part of the image to generalize quickly to other parts of the image, and we want similar effects for sequence data as well
- Each $x^{<t>}$ above is 10000 dimensional. Total input size = (max. # of words) x (10000), which is very large. ⇒ weight matrix of the 1st layer would have an enormous number of parameters.

- A RNN doesn't have either of the above disadvantages.

Recurrent Neural Networks



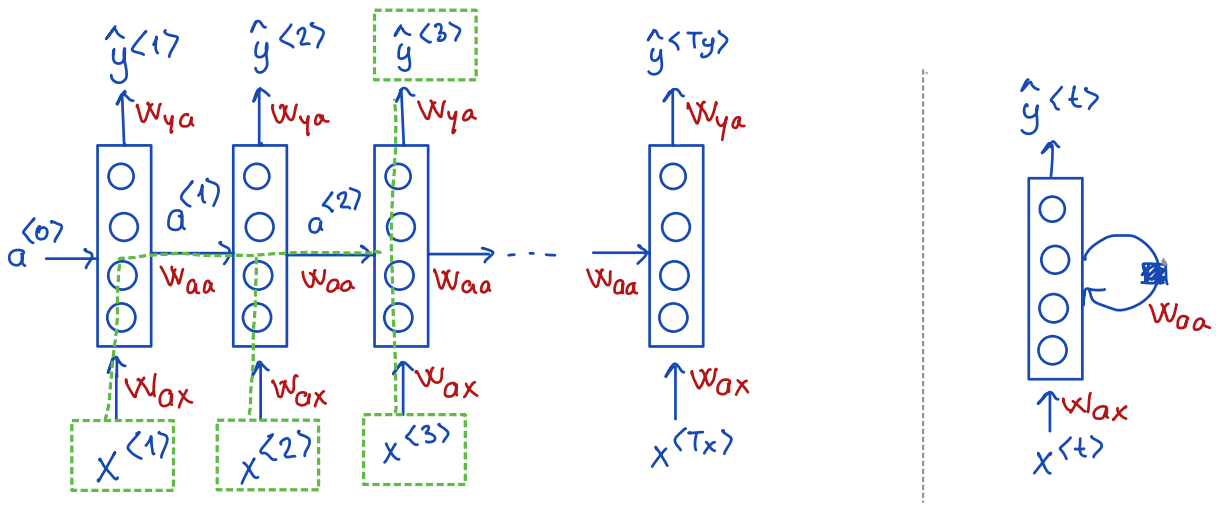
He said, “Teddy Roosevelt was a great President.”

He said, “Teddy bears are on sale!”

Andrew Ng

RNN:

- As you're reading the sentence from left to right, the first word you'll read is $x^{<1>}$.
- You take the $x^{<1>}$ and feed it into a neural network layer. That's a hidden layer of the first neural network, and it will try to predict the output $\hat{y}^{<1>}$ (is this part of a person's name or not)
- When you read the second word in the sentence $x^{<2>}$, instead of predicting $\hat{y}^{<2>}$ using only $x^{<2>}$, it also gets to input some information whatever is computed at time step 1.
- The activation value of time step 1 is passed on to time step 2.
- Next, RNN inputs $x^{<3>}$ and tries to produce $\hat{y}^{<3>}$ and so on.
- The parameters RNN uses for each time step are shared: W_{ax}, W_{aa}, W_{ya} show below.



- When making the prediction $\hat{y}^{<3>}$, the RNN gets the information not only from $x^{<3>}$, but also the information from $x^{<1>}$ and $x^{<2>}$. (the green dotted path in the above figure).
- A weakness of above RNN: It only uses the information that is earlier in the sequence to make a prediction. When predicting $\hat{y}^{<3>}$, it doesn't use any info from $x^{<4>}$, $x^{<5>}$ etc.

- This is a problem, e.g. given the sentence below, in order to decide whether or not the word Teddy is a part of person's name, it would be really useful to know not just info from first two words but also know info. from later words in the sentence.

He said, "Teddy Roosevelt was a great President."

- The sentence could have been:

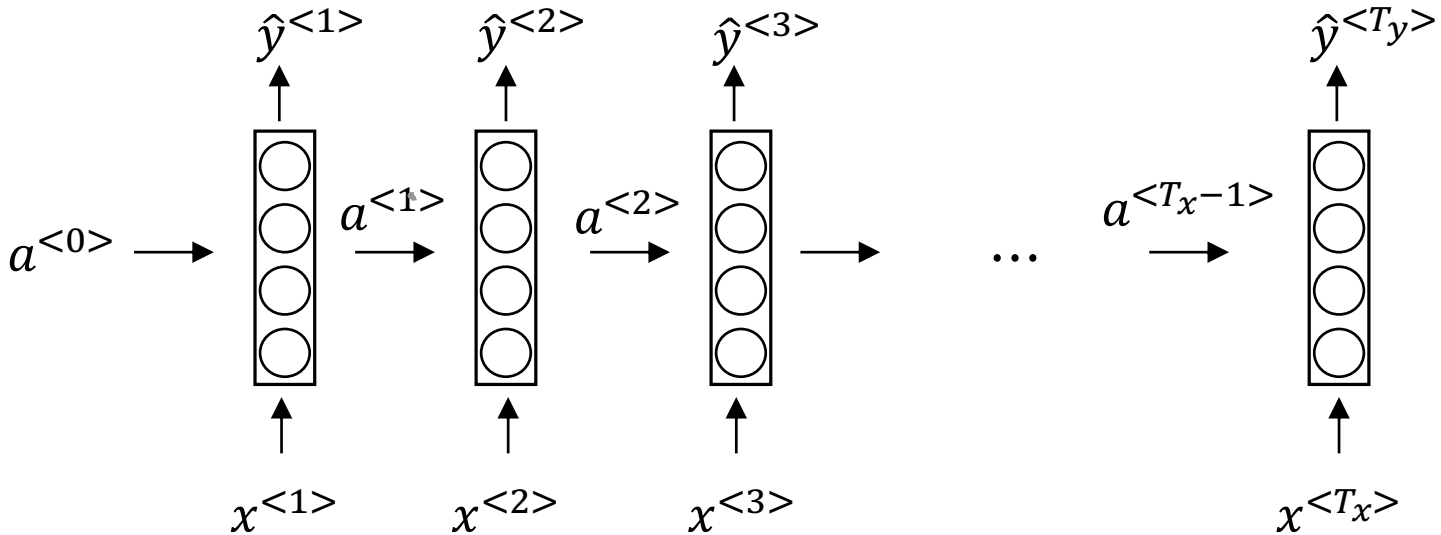
He said, "Teddy bears are on sale!"

So, given just the first 3 words, it is not possible to know for sure whether the word Teddy is part of a person's name.

- Bidirectional RNN (BRNN) will take into account information later in the sequence.

. Let's look at the calculations done in RNN.

Forward Propagation



$a^{<0>} = \vec{0}$

$a^{<1>} = g_1(W_{aa}\, a^{<0>} + W_{ax}\, x^{<1>} + b_a)$

$\hat{y}^{<1>} = g_2(W_{ya}\, a^{<1>} + b_y)$

tanh/ReLU

sigmoid/softmax (sigmoid for binary classification)

$$a^{<t>} = g(W_{aa}\, a^{<t-1>} + W_{ax}\, x^{<t>} + b_a)$$

$$\hat{y}^{<t>} = g(W_{ya}\, a^{<t>} + b_y)$$

Forward Propagation

Notation:

$W_{ax}\, x^{<1>}$

will be multiplied with x-like quantity

will compute a-like quantity

Andrew Ng

Simplified RNN notation

$$a^{<t>} = g(W_{aa}a^{<t-1>} + W_{ax}x^{<t>} + b_a)$$

(100,100) 100 (100,10000) 10000

simplify

parameters for computing activation

$$a^{\langle t \rangle} = g\left(W_a \left[a^{\langle t-1 \rangle}, x^{\langle t \rangle}\right] + b_a\right)$$

$$100 \updownarrow \left[\underset{100}{W_{aa}} \,\vdots\, \underset{10000}{W_{ax}}\right] = W_a$$: single parameter matrix

↳ (100, 10100)

$$\left[a^{\langle t-1 \rangle}, x^{\langle t \rangle}\right] = \begin{bmatrix} a^{\langle t-1 \rangle} \\ x^{\langle t \rangle} \end{bmatrix} \begin{matrix} \updownarrow 100 \\ \updownarrow 10000 \end{matrix}$$

(10100, 1)

$$\left[W_{aa} \,\vdots\, W_{ax}\right] \begin{bmatrix} a^{\langle t-1 \rangle} \\ x^{\langle t \rangle} \end{bmatrix} = W_{aa}\, a^{\langle t-1 \rangle} + W_{ax}\, x^{\langle t \rangle}$$

$$\hat{y}^{<t>} = g(W_{ya}a^{<t>} + b_y)$$

$$\hat{y}^{\langle t \rangle} = g\left(W_y a^{\langle t \rangle} + b_y\right)$$

parameters for computing output y

- Advantage of this notation: Rather than carrying around 2 parameter matrices, W_{aa} and W_{ax}, compress them to just 1 parameter matrix W_a.

Andrew Ng

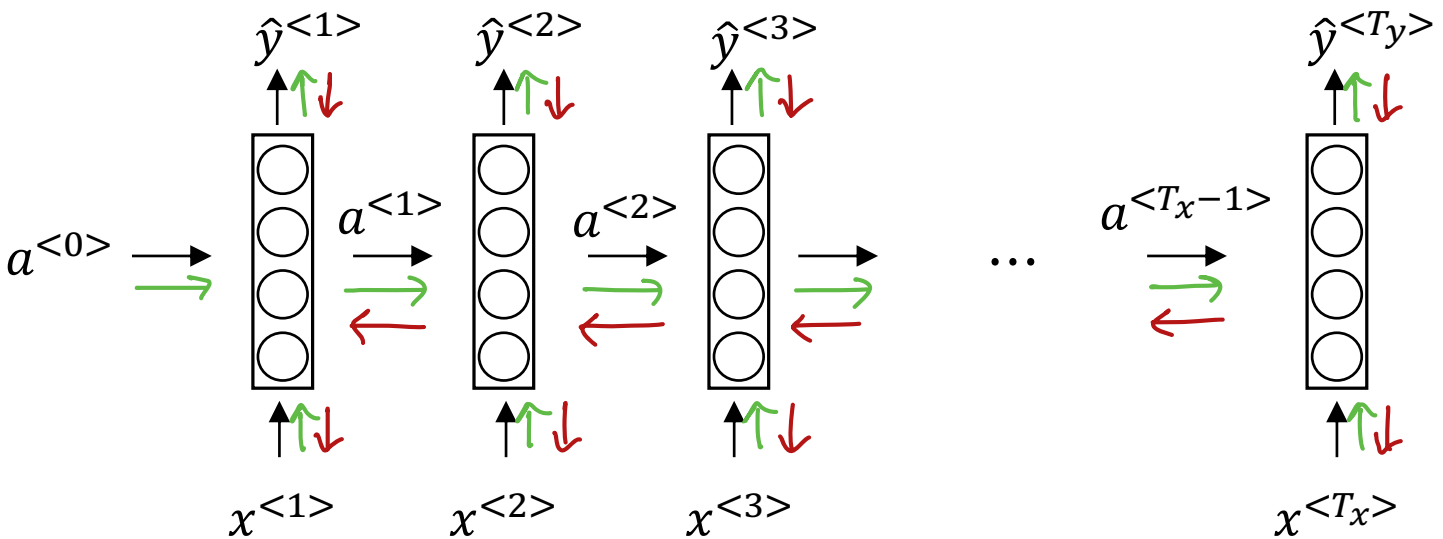


deeplearning.ai

Recurrent Neural Networks

Backpropagation through time

Forward propagation and backpropagation

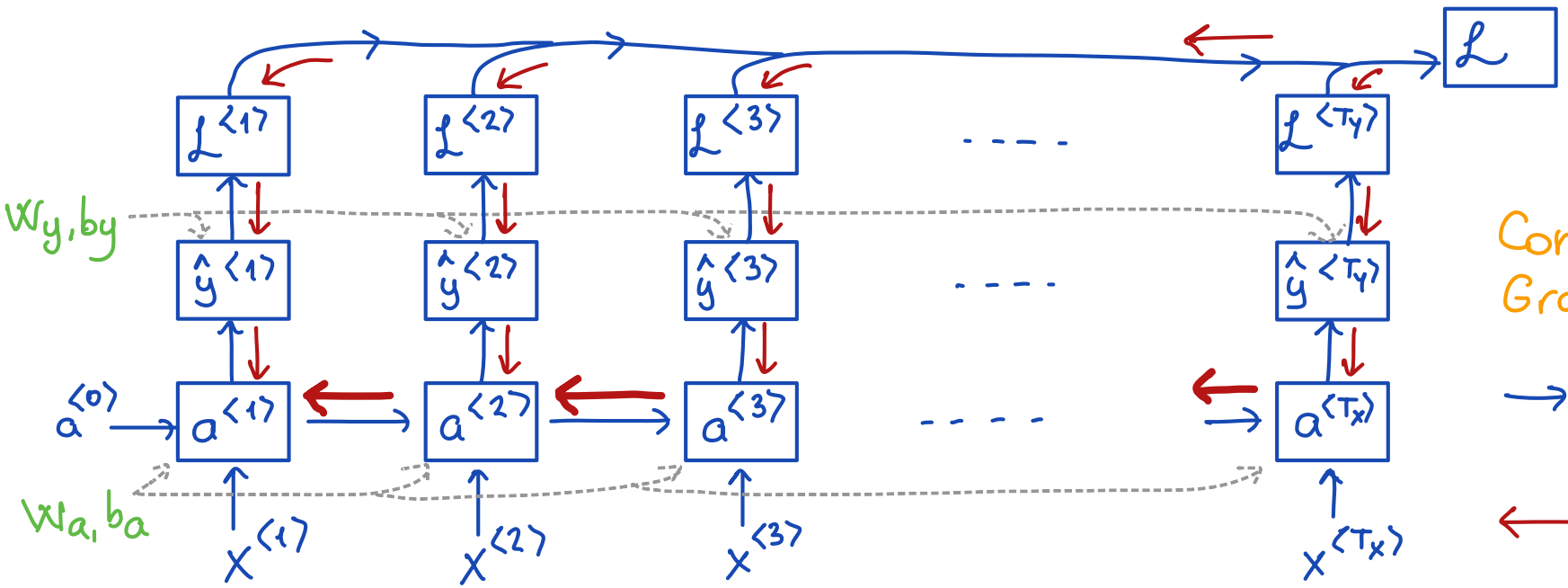


→ : Forward propagation calculations

← : Backpropagation calculations

Andrew Ng

Forward propagation and backpropagation



Computation Graph

→ : forward propagation

← : backward propagation

← : most significant recursive calculation

$$\mathcal{L}^{\langle t\rangle}(\hat{y}^{\langle t\rangle}, y^{\langle t\rangle}) = -y^{\langle t\rangle}\log\hat{y}^{\langle t\rangle} - (1-y^{\langle t\rangle})\log(1-y^{\langle t\rangle})$$

$$\mathcal{L}(\hat{y}, y) = \sum_{t=1}^{T_y} \mathcal{L}^{\langle t\rangle}(\hat{y}^{\langle t\rangle}, y^{\langle t\rangle})$$

↳ loss for the entire time sequence

Logistic regression loss for $\langle t\rangle$ (Binary cross-entropy)

Backpropagation through time

Andrew Ng

More Detailed Examples about Backpropagation in RNN:

- "Backpropagation in RNN, Backpropagation Through Time", Coding Lane, YouTube.

- Book: "Dive into Deep Learning" (d2l.ai)
  Section 9.7: Backpropagation Through Time



deeplearning.ai

Recurrent Neural Networks

Different types of RNNs

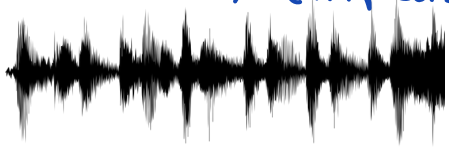
- So far, $T_x = T_y$ (the lengths of input and output sequences.
- However, this is not always the case in other applications.

Examples of sequence data

x (inputs)

y (outputs)
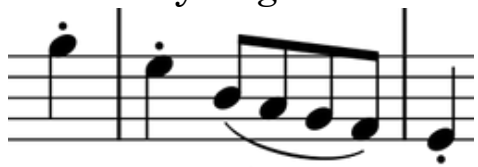
Speech recognition



→ "The quick brown fox jumped over the lazy dog."

Music generation

Ø $T_x = 0$

→ 

Sentiment classification

"There is nothing to like in this movie."

→ $T_y = 1$ ★☆☆☆☆

DNA sequence analysis

AGCCCCTGTGAGGAACTAG

→ AGCCCCTGTGAGGAACTAG

$T_x \neq T_y$

Machine translation

Voulez-vous chanter avec moi?

→ Do you want to sing with me?

Video activity recognition



→ Running

Name entity recognition

Yesterday, Harry Potter met Hermione Granger.

→ Yesterday, Harry Potter met Hermione Granger.

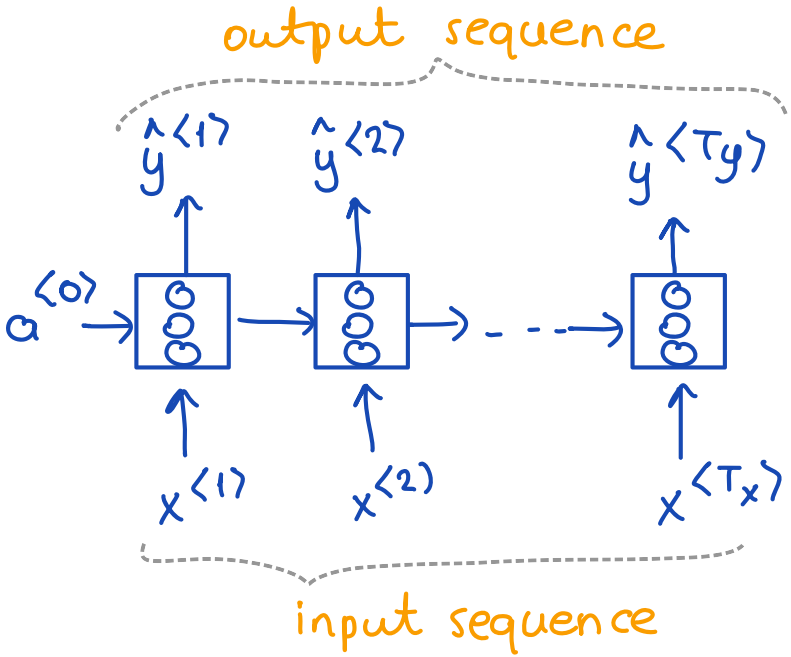
$T_x = T_y$

Andrew Ng

Examples of RNN architectures

Named entity recognition

$T_x = T_y$



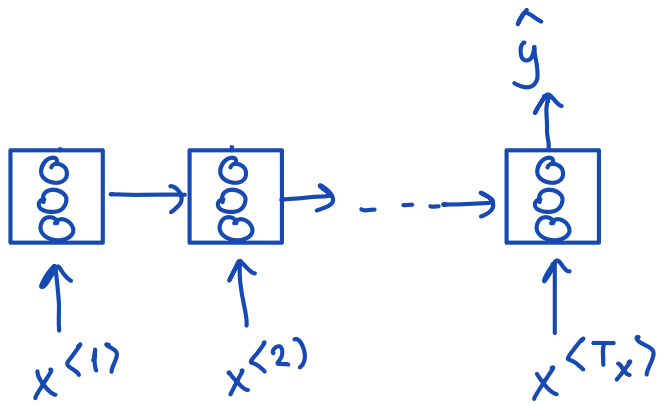
Many-to-many architecture

Sentiment Classification

X = text (e.g. movie review)

y = 0/1 or rating: 1...5

+ review, − review



Many-to-one architecture

Classical Neural Networks



one-to-one

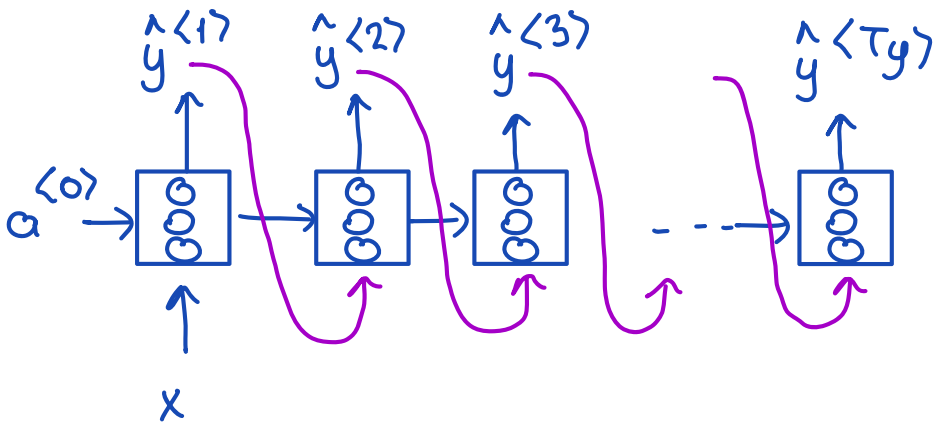
Andrew Ng

Examples of RNN architectures

Music Generation

$x \rightarrow y^{\langle 1 \rangle}\ y^{\langle 2 \rangle} \cdots y^{\langle T_y \rangle}$

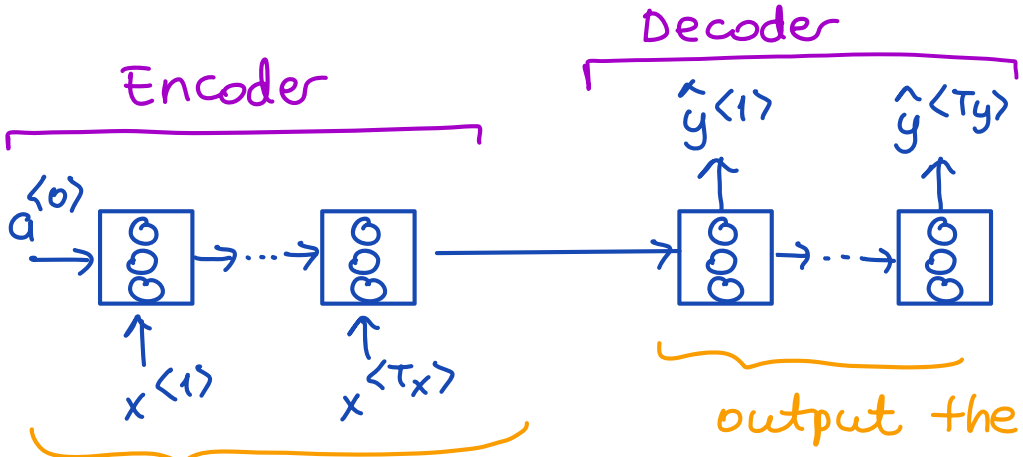
or $x = \emptyset$



One-to-many architecture

- You take the synthesized output $\hat{y}^{\langle t \rangle}$ and feed it to the next layer.

Machine Translation
(e.g. French to English)

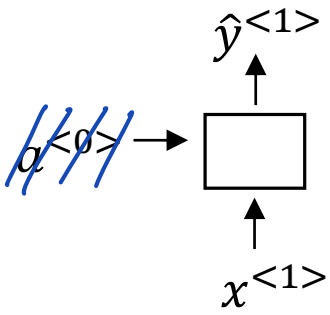


Many-to-many

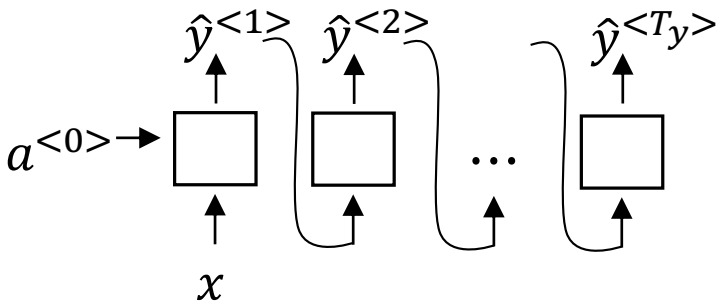
$T_x \neq T_y$

Andrew Ng

Summary of RNN types

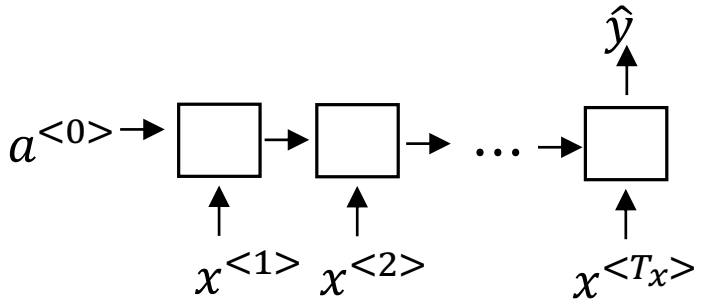


One to one



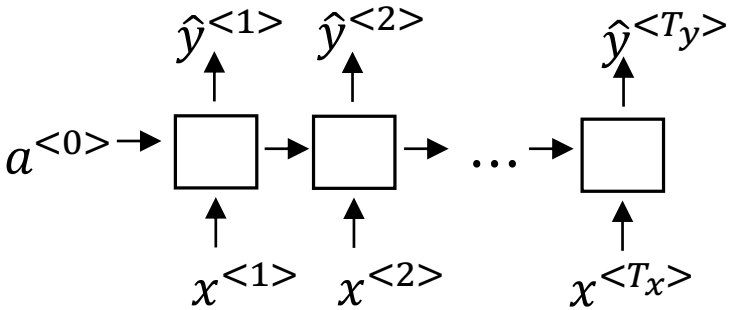
One to many

(music generation)



Many to one

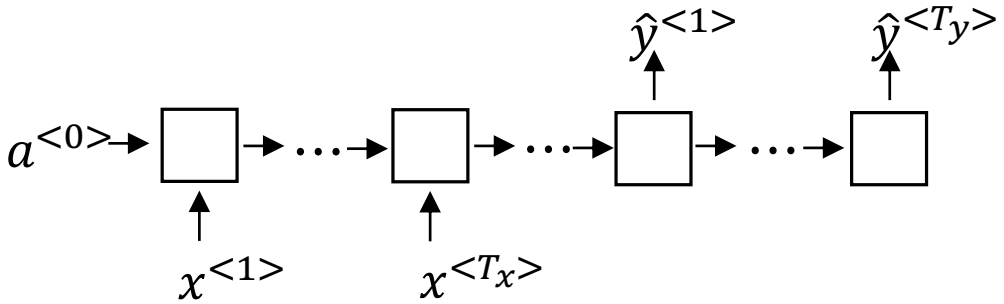
(Sentiment classification)



Many to many

(e.g. named entity recognition)

$T_x = T_y$



Many to many

(machine translation)

Andrew Ng



deeplearning.ai

Recurrent Neural Networks

Language model and sequence generation

- Language modeling is an important task in NLP.
- Goal: Build a language model using RNN.

What is language modelling?

Speech recognition

The apple and pair salad.

The apple and pear salad. → More Likely

- These sentences sound exactly the same.
- A language model picks the second sentence based on probability.

$P(\text{The apple and pair salad}) = 3.2 \times 10^{-13}$

$P(\text{The apple and pear salad}) = 5.7 \times 10^{-10}$ → more likely

$P(\text{sentence}) = ?$ ⟶ $P(y^{\langle 1 \rangle}, y^{\langle 2 \rangle}, \ldots, y^{\langle T_y \rangle}) = ?$ Prob. of a particular sequence of words

- Given a sentence, a language model tells the probability of the sentence:
  - what is the chance that the next sentence you read somewhere out there in the world (e.g. random newspaper, random email, random webpage etc.) will be a particular sentence?

Language modelling with an RNN

Training set: large corpus of english text.

Tokenize: Form a vocabulary and map each of the words to one-hot vectors or indices in your vocabulary.

ignore punctuation for now

Cats average 15 hours of sleep a day. <EOS>

$y^{<1>}$ $y^{<2>}$ $y^{<3>}$ . . . $y^{<8>}$ $y^{<9>}$

$x^{<t>} = y^{<t-1>}$: next slide

extra token for end-of-sentence may be added.

The Egyptian Mau is a bread of cat. <EOS>

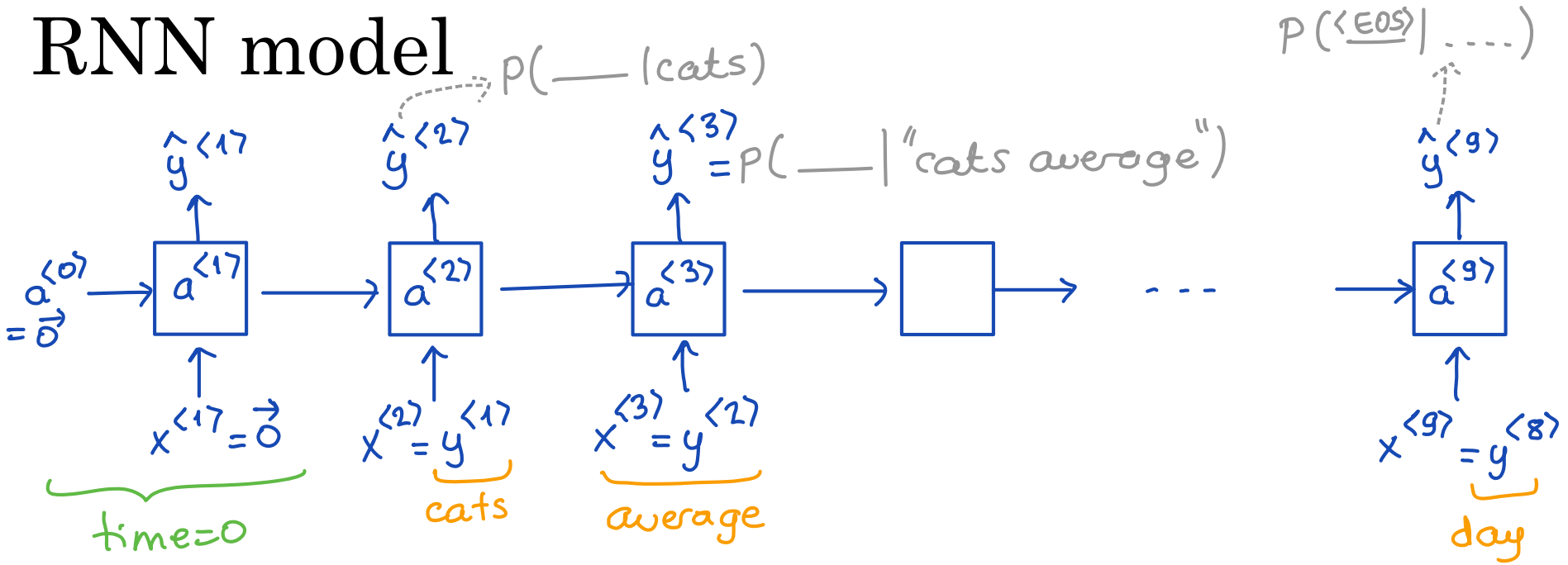
<UNK>

token for unknown words, which are not in the vocabulary.

Andrew Ng

- Next: Build an RNN model to to compute the chance of these different sequences.

RNN model



Cats average 15 hours of sleep a day. <EOS>

$$\mathcal{L}(\hat{y}^{<t>}, y^{<t>}) = -\sum_i y_i^{<t>} \log \hat{y}_i^{<t>}$$ → softmax loss

$$\mathcal{L} = \sum_t \mathcal{L}^{<t>}(\hat{y}^{<t>}, y^{<t>})$$ → overall loss.

Andrew Ng

- At time=1, you'll compute the activation $a^{<1>}$, which will make a softmax prediction to try to figure out what's the probability of the first word y (i.e $\hat{y}^{<1>}$):
  → what's the chance that the first word is "a": P(a)
  " " " " " " "aaron": P(aaron)
  " " " " " " "cats": P(cats)
  " " " " " " "zulu": P(zulu)
  " " " " " " P(<UNK>)
  P(<EOS>)

  10002 way softmax output

- At time=2, RNN will try to figure out what the second word is. But we will also give it the correct first word (cats).
  - The RNN will predict the probability of any word in the dictionary given the first word.
  - Right answer: P(average|cats)

- At time=3, RNN will compute $a^{<3>}$, to predict what the third word is given the first two words ("cats average").
  - RNN will predict the probability of any word in the dictionary given the first two words ("cats average").
  - The right answer is "15", that is P("15" | "cats average") should be the highest probability output of the softmax unit.

- RNN learns to predict one word at a time going from left to right, given the previous words.

- To train the RNN, we need to define a cost function.

$$\mathcal{L}^{\langle t \rangle}(\hat{y}^{\langle t \rangle}, y^{\langle t \rangle}) = -\sum_i y_i^{\langle t \rangle} \log \hat{y}_i^{\langle t \rangle} \quad \rightsquigarrow \text{softmax loss}$$

($\hat{y}^{\langle t \rangle}$: predicted, $y^{\langle t \rangle}$: true)

$$\mathcal{L} = \sum_t \mathcal{L}^{\langle t \rangle}(\hat{y}^{\langle t \rangle}, y^{\langle t \rangle})$$

- Given a sentence with 3 words $(y^{\langle 1 \rangle}, y^{\langle 2 \rangle}, y^{\langle 3 \rangle})$ the way you can figure out what the chance of this entire sentence would be:

$$P(y^{\langle 1 \rangle}, y^{\langle 2 \rangle}, y^{\langle 3 \rangle}) = \underbrace{P(y^{\langle 1 \rangle})}_{\text{output of first softmax}} P(y^{\langle 2 \rangle} \mid y^{\langle 1 \rangle}) P(y^{\langle 3 \rangle} \mid y^{\langle 1 \rangle}, y^{\langle 2 \rangle})$$



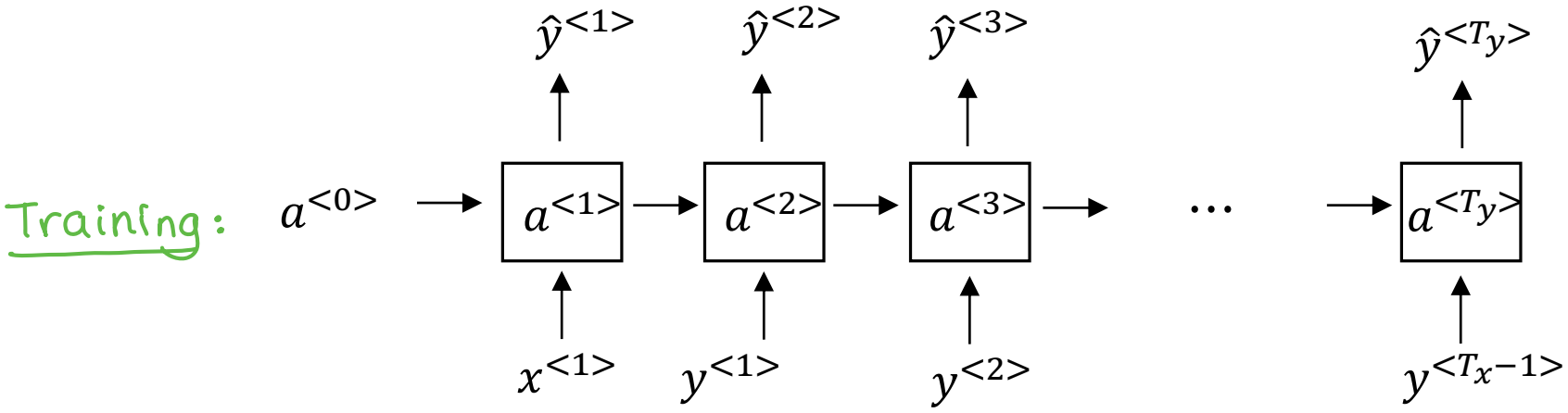
deeplearning.ai

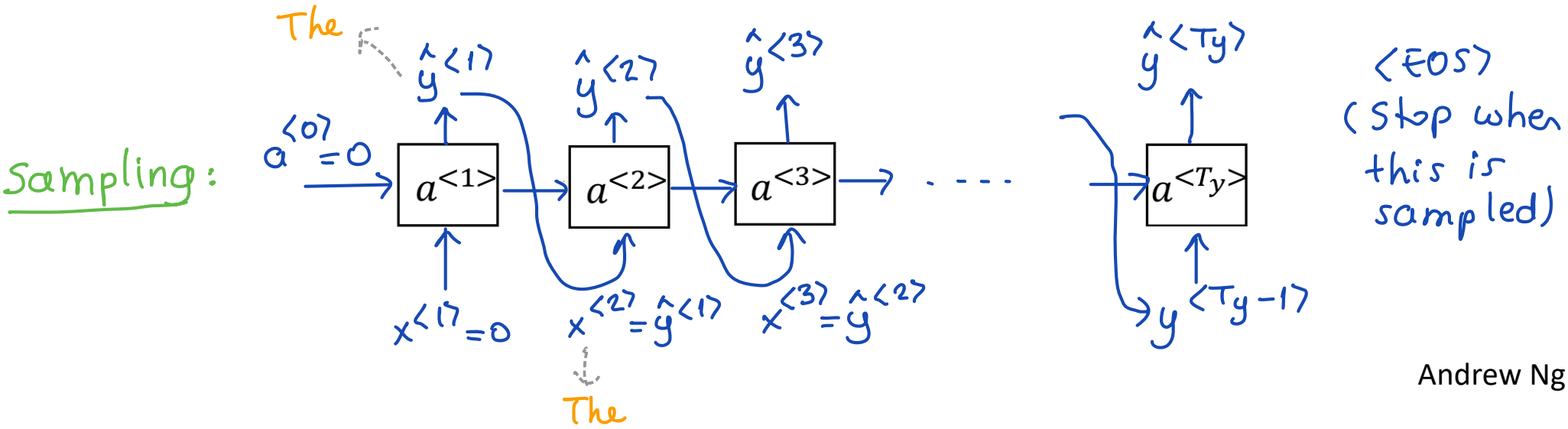
Recurrent Neural Networks

Sampling novel sequences

- After you train a sequence model, one of the ways you can informally get a sense of what is learned is to sample novel sequences.

Sampling a sequence from a trained RNN





Andrew Ng

- A sequence model, models the chance of any particular sequence of words: $P(y^{<1>}, \ldots y^{<Ty>})$. We would like to sample from this distribution.
- Sampling the first word: The first softmax output is a pdf over all possible words in the vocabulary:
  P(a), P(aaron) ... P(zulu), P(<UNK>)
  Sample from this distribution using e.g: np.random.choice
  After sampling the first word output will be $\hat{y}^{<1>}$
- Sampling the second word: Take the selected first word and use it as the input to generate probability of any word given the first word. Sample from this distribution to generate the second word.
- Continue sampling words as described above.
- How do we know when the sequence ends?
  - → You could keep sampling until <EOS> (end of sentence) is sampled. (If it is in your vocabulary)
  - → OR You can sample a predefined number of words
  - → If <UNK> (unknown token) is generated during sampling, you can just reject it and resample from the rest of the vocabulary.

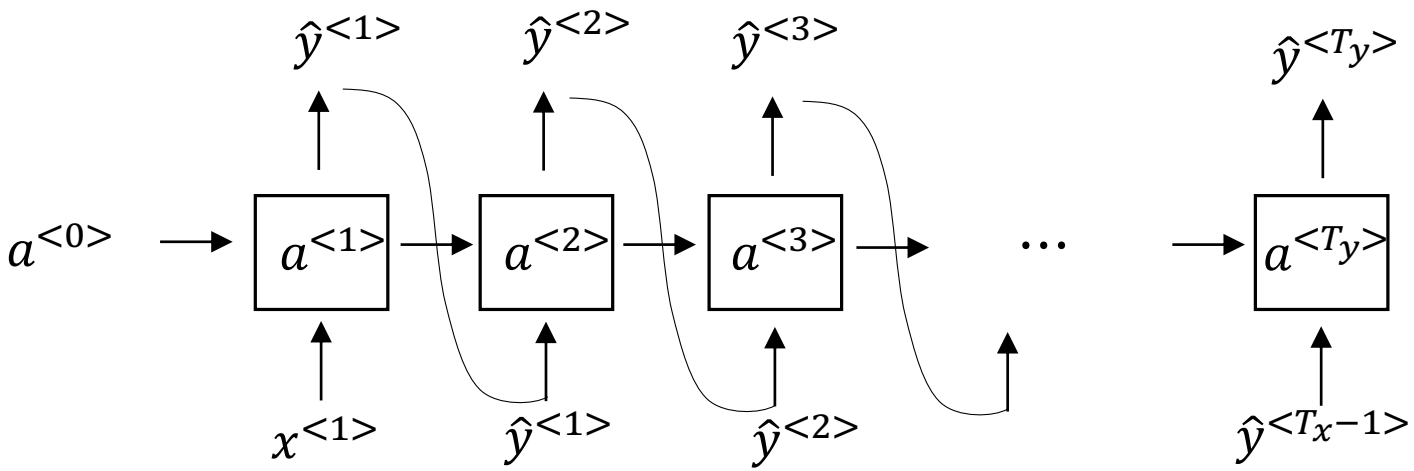
Character-level language model

- we have been building word-level RNN, where the vocabulary are words.

Vocabulary = [a, aaron, …, zulu, <UNK>]

- we can also build a character level RNN. where vocabulary contains the alphabet, digits, punctuation marks:

Vocabulary = [a, b, c, ---, z, ␣, ., ;, i, 0, ---, 9, A, ---, Z]



$\hat{y}^{<1>}$ $\hat{y}^{<2>}$ $\hat{y}^{<3>}$ $\hat{y}^{<4>}$ ---

cat average

Andrew Ng

- Using character-level language model has some pros (+) and cons (-).
  - (+) You don't have to worry about <UNK> word tokens. A character-level language model is able to assign any word a non-zero probability.
  - (-) You end up with much longer sequences. Many English sentences will have 10-20 words but have many dozens of characters. Hence, character-level language models are not as good as word level language models at capturing long-range dependencies between how the earlier parts of the sentence also affect later parts of the sentence
  - (-) Character-level language models are computationally more expensive to train.

- Examples sampled from a character-level language model.

Sequence generation

News

President enrique peña nieto, announced sench's sulk former coming football langston paring.

"I was not at all surprised," said hich langston.

"Concussion epidemic", to be examined.

The gray football the told some and this has on the uefa icon, should money as.

From a model trained by news articles.

Shakespeare

The mortal moon hath her eclipse in love.

And subject of this thou art another this fold.

When besser be my love to me see sabl's.

For whose are ruse of mine eyes heaves.

From a model trained with Shakespeare.

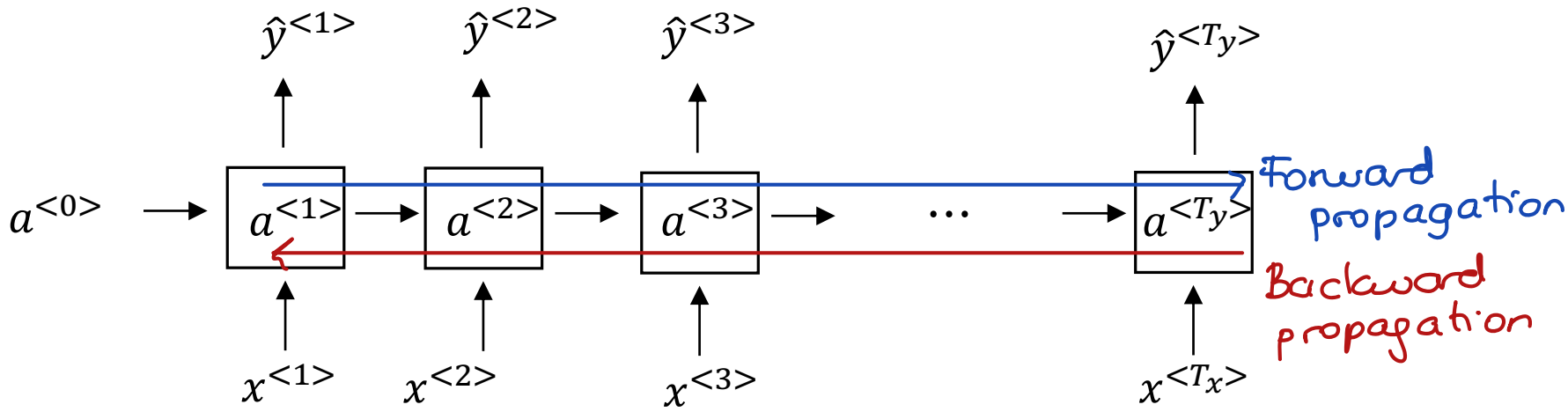


deeplearning.ai

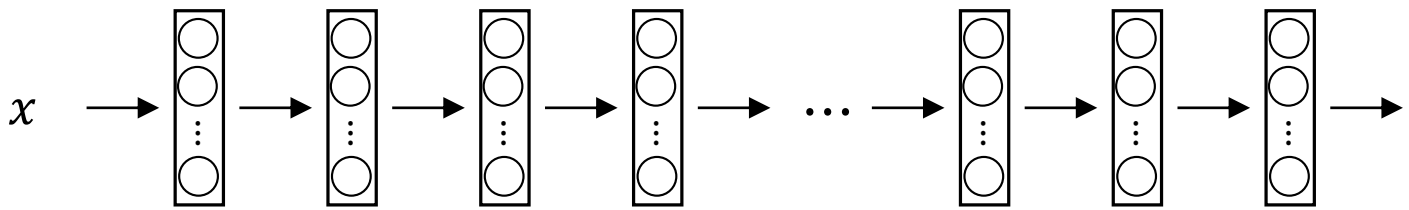
Recurrent Neural Networks

Vanishing gradients with RNNs

Vanishing gradients with RNNs



DNN:



100 layers

forward propagation

Backward propagation

Exploding gradients.

Andrew Ng

- One of the problems of the basic RNN algorithm is that it runs into vanishing gradient problems.
- Example: Language modeling

  The cat, which actually ate a bunch of food which..., was full.

  The cats, which actually ate - - - - , were full.
- This is one example of when language can have very long-term dependencies, where a word (cat/cats) much earlier can affect what needs to come (was/were) much later in the sequence.
- Basic RNN we have seen so far is not very good at capturing very long-term dependencies.
- In RNN forward propagation goes from left to right and backward propagation goes from right to left. We have a vanishing gradient problem.
- In RNN it is difficult for the computations at later steps to affect the computations that are earlier (during backpropagation)
- It is difficult for RNN to memorize the singular or plural noun (cat vs. cats) so that later in the sequence it can generate either was or were depending on whether it was singular or plural.
- The basic RNN model has many local influences, meaning that e.g. $\hat{y}^{\langle 3 \rangle}$ is influenced mainy by inputs that are somewhat close ($x^{\langle 3 \rangle}$, $x^{\langle 2 \rangle}$, $x^{\langle 1 \rangle}$).
- It is difficult for a later output to be strongly influenced by an input that was very early in the sequence.
- This is a weakness of the RNN algorithm.
- Another problem: Exploding gradients can be catastrophic because the exponentially large gradients can cause the parameters to become very large.

- If you see exploding gradients (e.g. NaN (not a number)), one solution is to apply gradients clipping (i.e. look at the gradient vectors, and if it is bigger than a threshold, re-scale the gradient vectors so that it's not so big, that is clipped according to some maximum value.
- But vanishing gradients is much harder to solve.
- An RNN processing data over 1000 time stamps, or over 10000 time stamps are basically 1000 layer or 10000 layer neural networks.

- NEXT: GRU's (gated recurrent units) is an effective solution for addressing the vanishing gradient problem, which will allow the neural network to capture much longer dependencies.

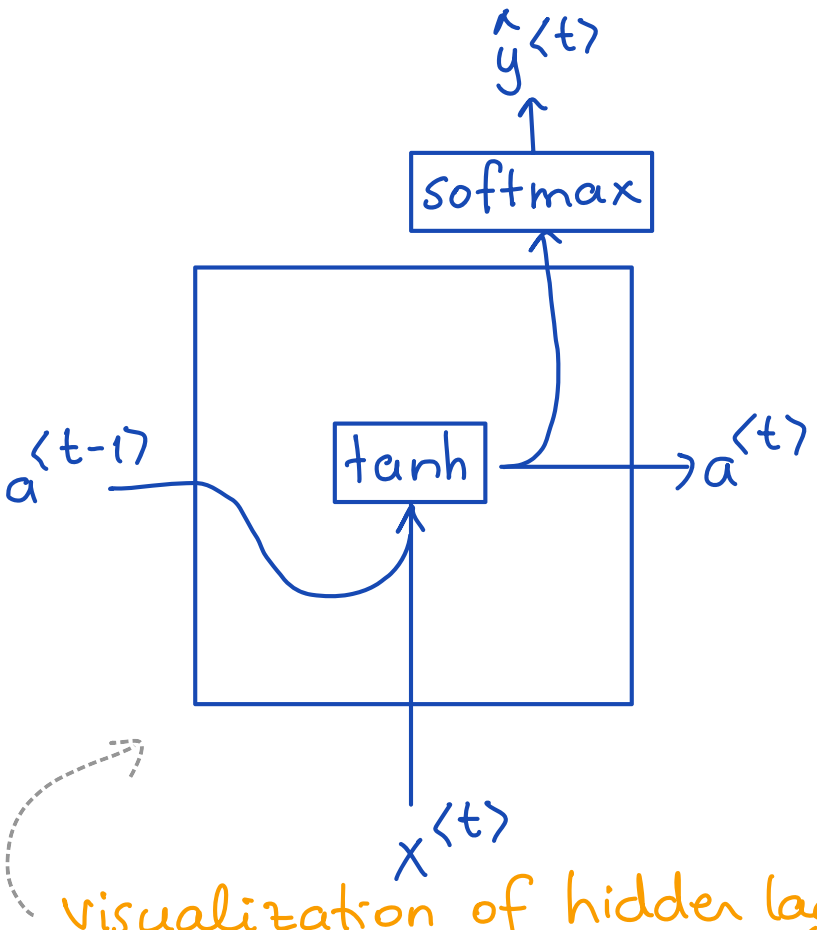


deeplearning.ai

Recurrent Neural Networks

Gated Recurrent Unit (GRU)

RNN unit: a modification to the RNN hidden layer that makes it much better at capturing long-range connections and helps a lot with the vanishing gradient problem.



Visualization of hidden layer of RNN

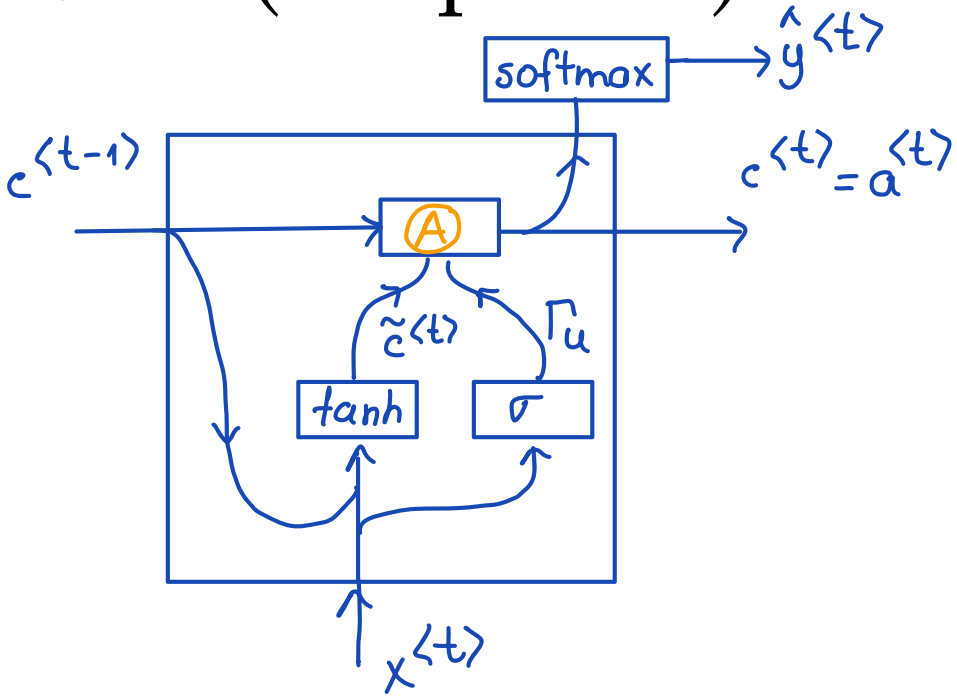
tanh

$$a^{<t>} = g(W_a[a^{<t-1>}, x^{<t>}] + b_a)$$

Activation at time t of RNN.

Andrew Ng

GRU (simplified)



c : memory cell

$c^{\langle t \rangle} = a^{\langle t \rangle}$

$$\tilde{c}^{\langle t \rangle} = \tanh\left(W_c\left[c^{\langle t-1 \rangle}, x^{\langle t \rangle}\right] + b_c\right)$$

$$\Gamma_u = \sigma\left(W_u\left[c^{\langle t-1 \rangle}, x^{\langle t \rangle}\right] + b_u\right)$$

↳ update gate in $[0, 1]$

σ : sigmoid function.

(A) $$c^{\langle t \rangle} = \Gamma_u * \tilde{c}^{\langle t \rangle} + (1 - \Gamma_u) * c^{\langle t-1 \rangle}$$

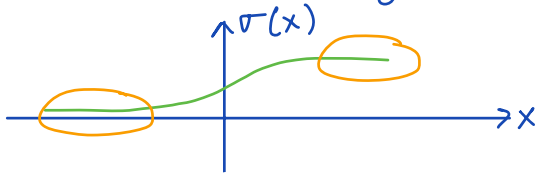
elementwise

singular

The cat, which already ate …, was full.

[Cho et al., 2014. On the properties of neural machine translation: Encoder-decoder approaches]
[Chung et al., 2014. Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling]

- In the sentence "The cat, which already ate ---, was full.", we would like to remember whether cat was singular or plural.
- The GRU unit has a new variable "c", which stands for cell, for memory cell.
- The memory cell will help us remember whether cat was singular or plural as we gets much further into the sentence.
- $\tilde{c}^{\langle t \rangle}$: candidate for replacing $c^{\langle t \rangle}$: $\tilde{c}^{\langle t \rangle} = \tanh(W_c[c^{\langle t-1 \rangle}, x^{\langle t \rangle}] + b_c)$
  At each time step we will consider overwriting the memory cell $c^{\langle t \rangle}$ with $\tilde{c}^{\langle t \rangle}$.
- Important idea of GRU is the gate : $\Gamma_u = \sigma(W_u[c^{\langle t-1 \rangle}, x^{\langle t \rangle}] + b_u)$ ↳gate
  where σ is the sigmoid function.



For most of the input values the sigmoid function takes values very close to 1 or 0. So it takes the value 0 or 1 most of the time.

c : memory cell

$c^{\langle t \rangle} = a^{\langle t \rangle}$

$\tilde{c}^{\langle t \rangle} = \tanh(W_c[c^{\langle t-1 \rangle}, x^{\langle t \rangle}] + b_c)$: candidate for replacing $c^{\langle t \rangle}$

$\Gamma_u = \sigma(W_u[c^{\langle t-1 \rangle}, x^{\langle t \rangle}] + b_u)$

↳update gate in $[0,1]$

σ : sigmoid function.

$c^{\langle t \rangle} = \Gamma_u * \tilde{c}^{\langle t \rangle} + (1 - \Gamma_u) * c^{\langle t-1 \rangle}$: gate will decide whether or not $c^{\langle t \rangle}$ is updated.

↳ elementwise product

$$c^{\langle t \rangle} = \begin{cases} \tilde{c}^{\langle t \rangle} & \text{if } \Gamma_u \cong 1 \quad \text{(update the memory cell to memorize something)} \\ c^{\langle t-1 \rangle} & \text{if } \Gamma_u \cong 0 \quad \text{(don't need to update)} \end{cases}$$

good time to update the memory cell, since talking about a new concept, the subject of the sentence.

$\Gamma_u = 1, \Gamma_u = 0, \Gamma_u = 0 \; - - - - - - - \ldots$

$c^{\langle t \rangle} = 1 \; - - - - - - - - - - - - - = 1$

- The cat, which already ate ..., was full

- Maybe the memory cell $c^{\langle t \rangle}$ (or a bit in it) will be set to 0 or 1, depending on whether we're considering, i.e. the subject of the sentence is plural or not.
- $c^{\langle t \rangle} = 1$ should memorize that "cat" is singular all the way up to "was". The job of the gate, Γ_u, is to decide when to update this value.
- GRU can learn long-term dependencies better than RNN.
- If $\Gamma_u = 0$, then $c^{\langle t \rangle} = c^{\langle t-1 \rangle}$, i.e, GRU is very good at maintaining the value of the cell. Hence, it doesn't suffer much from the vanishing gradient problem.
- Implementation details:

  $c^{\langle t \rangle}$: a vector (i.e. 100 dim.)

  $\Rightarrow \tilde{c}^{\langle t \rangle}$: 100 dimensional, and

  Γ_u : 100 dim, the values are mostly 0 or 1, telling which elements of the memory cell vector will be updated

  e.g. one element (or bit) can be used to remember "cat", another element to remember another concept etc.
- You can use different bits and change only a subset of bits at every point in time.
- The above was a simplified version of GRU.

Full GRU

→ candidate

$\tilde{h}$ $\tilde{c}^{<t>} = \tanh(W_c[\, c^{<t-1>}, x^{<t>}] + b_c)$ ⟶ $\tilde{c}^{\langle t \rangle} = \tanh(W_c[\Gamma_r * c^{\langle t-1 \rangle}, x^{\langle t \rangle}] + b_c)$

u $\Gamma_u = \sigma(W_u[\, c^{<t-1>}, x^{<t>}] + b_u)$ (update gate)

r $\Gamma_r = \sigma(W_r[c^{\langle t-1 \rangle}, x^{\langle t \rangle}] + b_r)$ (relevance gate)

relevance

How relevant is $c^{\langle t-1 \rangle}$ for computing $\tilde{c}^{\langle t \rangle}$ (the next candidate for $c^{\langle t \rangle}$)?

h $c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) + c^{<t-1>}$

↳ memory cell

alternative notation used in the literature.

The cat, which ate already, was full.

- The above version of GRU has been observed to be better in modelling long-range dependencies and also address vanishing gradient problems.

Andrew Ng



deeplearning.ai

Recurrent Neural Networks

LSTM (long short term memory) unit

- The other type of unit that allows us to learn very long range dependencies in a sequence is the LSTM (long short term memory units.)
  - LSTM is more powerful than GRU.
  - For LSTM $a^{\langle t \rangle} \neq c^{\langle t \rangle}$

GRU and LSTM

GRU

$$\tilde{c}^{<t>} = \tanh(W_c[\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$
↳ candidate

$$\Gamma_u = \sigma(W_u[c^{<t-1>}, x^{<t>}] + b_u)$$
↳ update gate

$$\Gamma_r = \sigma(W_r[c^{<t-1>}, x^{<t>}] + b_r)$$
↳ relevance gate

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$
↳ memory cell

$$a^{<t>} = c^{<t>}$$

[Hochreiter & Schmidhuber 1997. Long short-term memory]

LSTM ($a^{<t>} \neq c^{<t>}$)

$$\tilde{c}^{<t>} = \tanh(W_c[a^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u[a^{<t-1>}, x^{<t>}] + b_u)$$
↳ update gate

$$\Gamma_f = \sigma(W_f[a^{<t-1>}, x^{<t>}] + b_f)$$
↳ forget gate

$$\Gamma_o = \sigma(W_o[a^{<t-1>}, x^{<t>}] + b_o)$$
↳ output gate

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}$$

$$a^{<t>} = \Gamma_o * \tanh c^{<t>}$$

option of keeping the old value $c^{<t-1>}$ and adding to it the new value $\tilde{c}^{<t>}$.

LSTM units

GRU

$$\tilde{c}^{<t>} = \tanh(W_c[\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u[\, c^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_r = \sigma(W_r[\, c^{<t-1>}, x^{<t>}] + b_r)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

[Hochreiter & Schmidhuber 1997. Long short-term memory]

LSTM

$$\tilde{c}^{<t>} = \tanh(W_c[a^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u[\, a^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_f = \sigma(W_f[\, a^{<t-1>}, x^{<t>}] + b_f)$$

$$\Gamma_o = \sigma(W_o[\, a^{<t-1>}, x^{<t>}] + b_o)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}$$

$$a^{<t>} = \Gamma_o * c^{<t>} \quad = \Gamma_o * \tanh(c^{<t>})$$

tanh

Andrew Ng

LSTM in pictures

$$\tilde{c}^{<t>} = \tanh(W_c[a^{<t-1>}, x^{<t>}] + b_c)$$

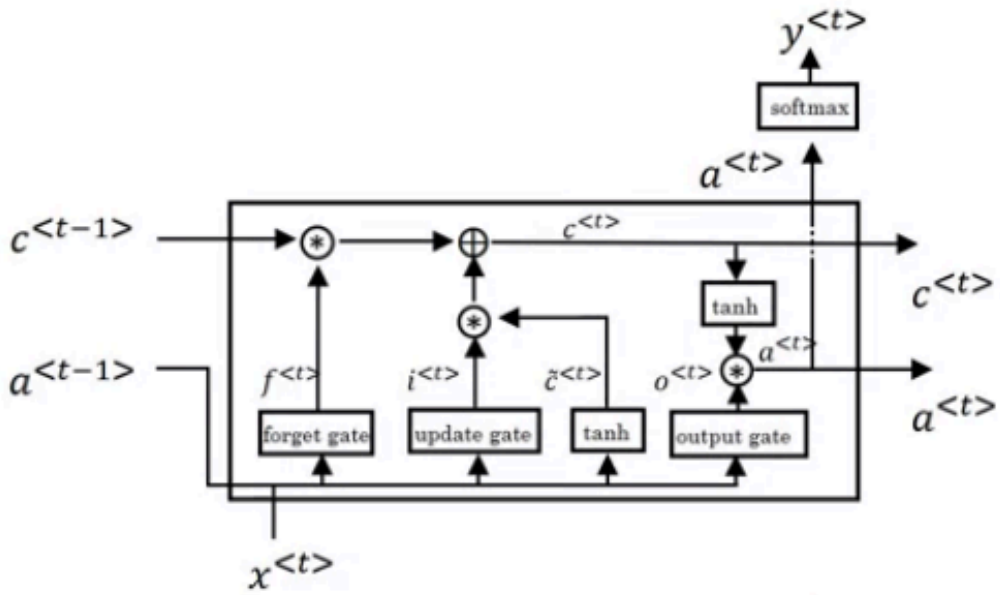
$$\Gamma_u = \sigma(W_u[a^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_f = \sigma(W_f[a^{<t-1>}, x^{<t>}] + b_f)$$

$$\Gamma_o = \sigma(W_o[a^{<t-1>}, x^{<t>}] + b_o)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}$$

$$a^{<t>} = \Gamma_o * \tanh c^{<t>}$$



Andrew Ng

LSTM in pictures

$$\tilde{c}^{<t>} = \tanh(W_c[a^{<t-1>}, x^{<t>}] + b_c)$$

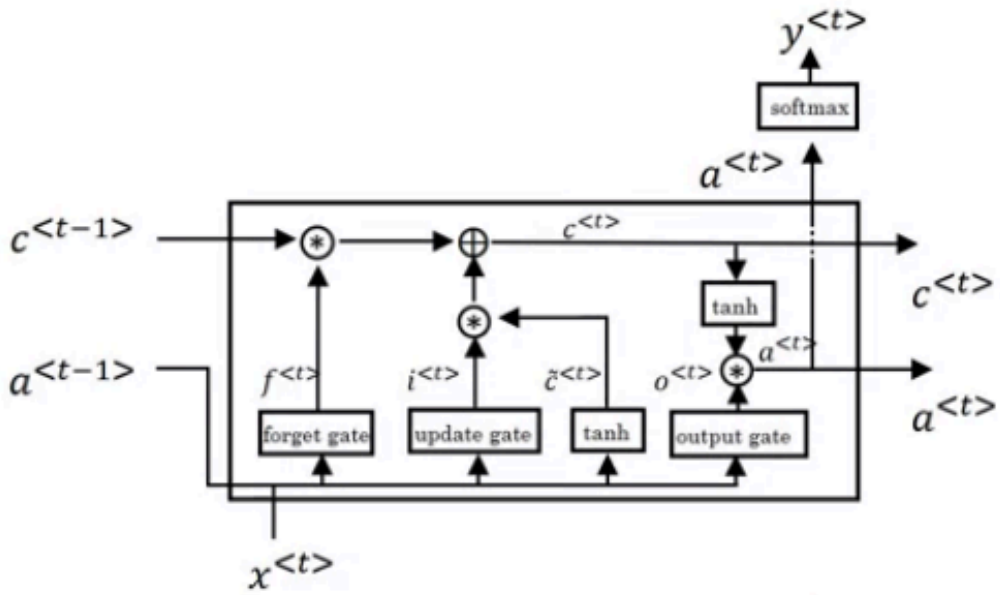
$$\Gamma_u = \sigma(W_u[a^{<t-1>}, x^{<t>}] + b_u)$$

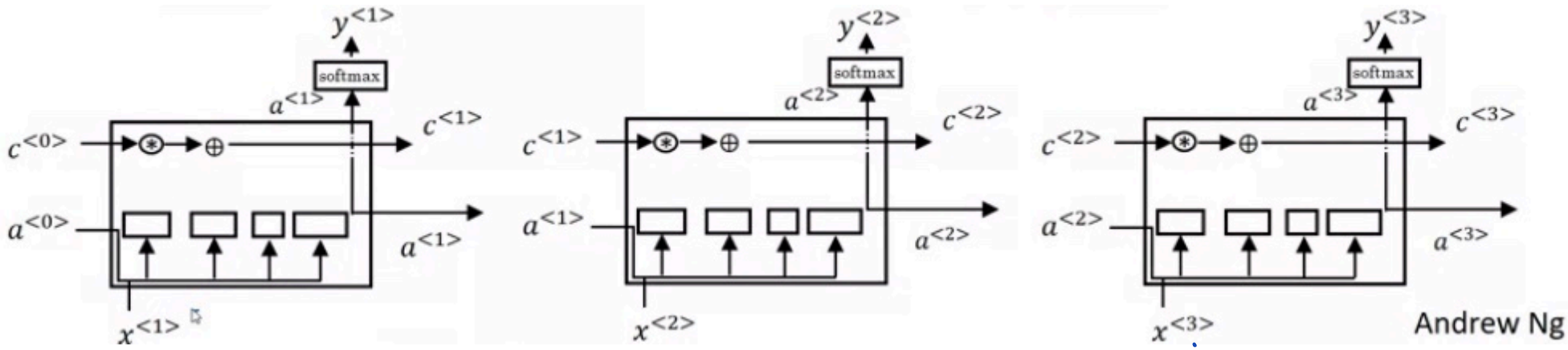
$$\Gamma_f = \sigma(W_f[a^{<t-1>}, x^{<t>}] + b_f)$$

$$\Gamma_o = \sigma(W_o[a^{<t-1>}, x^{<t>}] + b_o)$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}$$

$$a^{<t>} = \Gamma_o * \tanh c^{<t>}$$





- Connect a bunch of LSTM units in parallel.

LSTM in pictures

$$\tilde{c}^{<t>} = \tanh(W_c[a^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u[a^{<t-1>}, x^{<t>}] + b_u)$$

$$\Gamma_f = \sigma(W_f[a^{<t-1>}, x^{<t>}] + b_f)$$

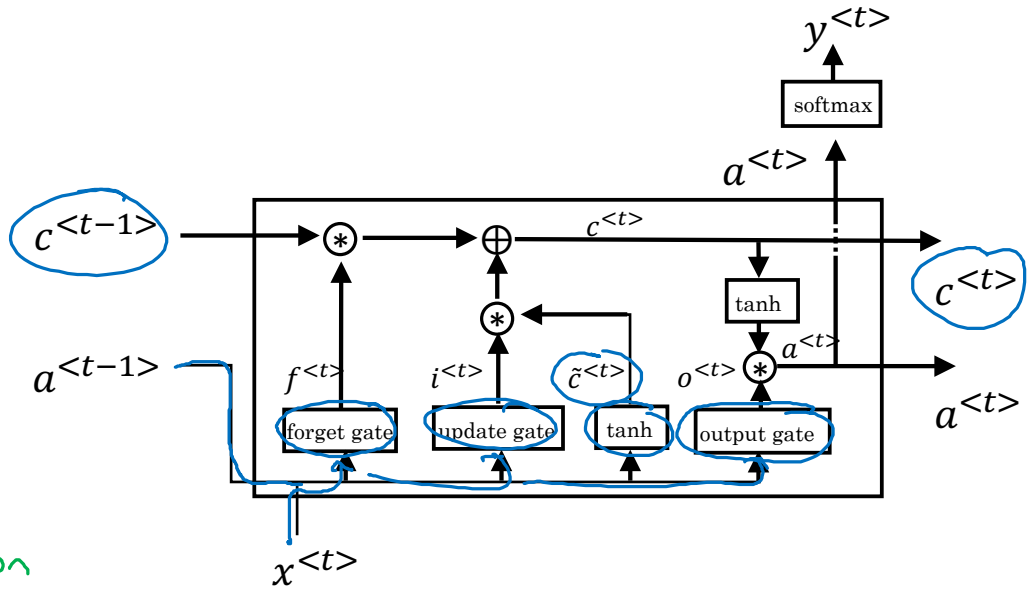
$$\Gamma_o = \sigma(W_o[a^{<t-1>}, x^{<t>}] + b_o)$$

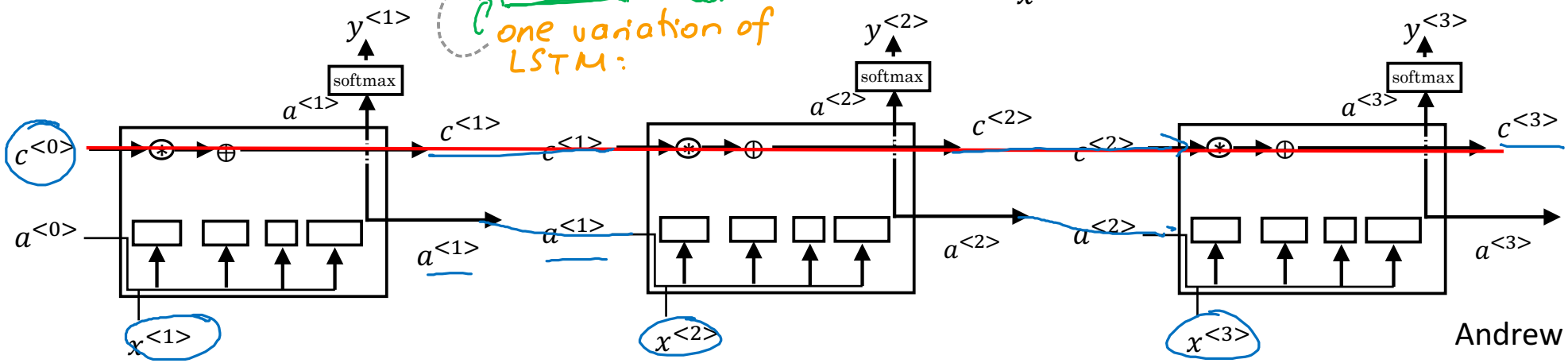
$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}$$

$$a^{<t>} = \Gamma_o * c^{<t>}$$

peephole connection

one variation of LSTM:





Andrew Ng

- The red line at the top (previous slide) shows that as long as you set the forget and the update gates appropriately, it is relatively easy for the LSTM to have some value $c^{\langle 0 \rangle}$.
- LSTM is very good at memorizing certain values for a long time.
- Even for a long time, certain real values can be stored in the memory cells for many time steps.
- One variation of LSTM is <u>peephole connection</u>: add $c^{\langle t-1 \rangle}$ to the equations above as shown in green.

- When do we use a GRU and when do we use an LSTM?
  - → There isn't a widespread consensus on this.
  - → LSTM came much earlier.
  - → GRU was invented as a simplification of the much complicated LSTM model.
  - → GRU is simpler and easier to build deep networks, and computationally faster.
  - → LSTM is more powerful (it has 3 gates) and flexible.
  - → Most people use LSTM instead of GRU.



deeplearning.ai

Recurrent Neural Networks

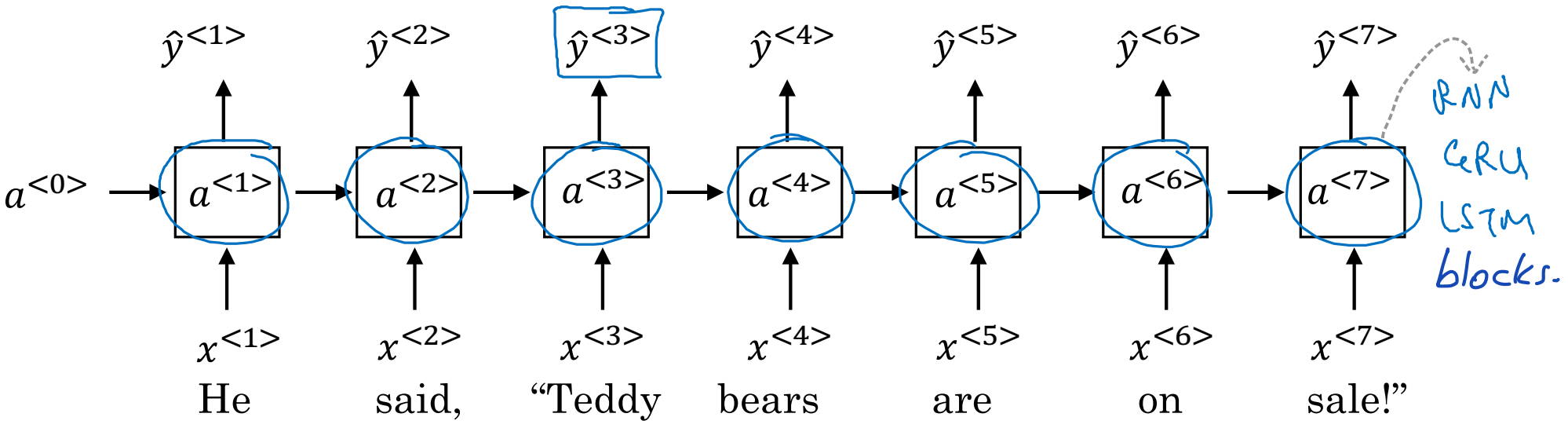
Bidirectional RNN

- Goal: Get information from both earlier and later in the sequence, together with the information from the present.

Getting information from the future

He said, “Teddy bears are on sale!”

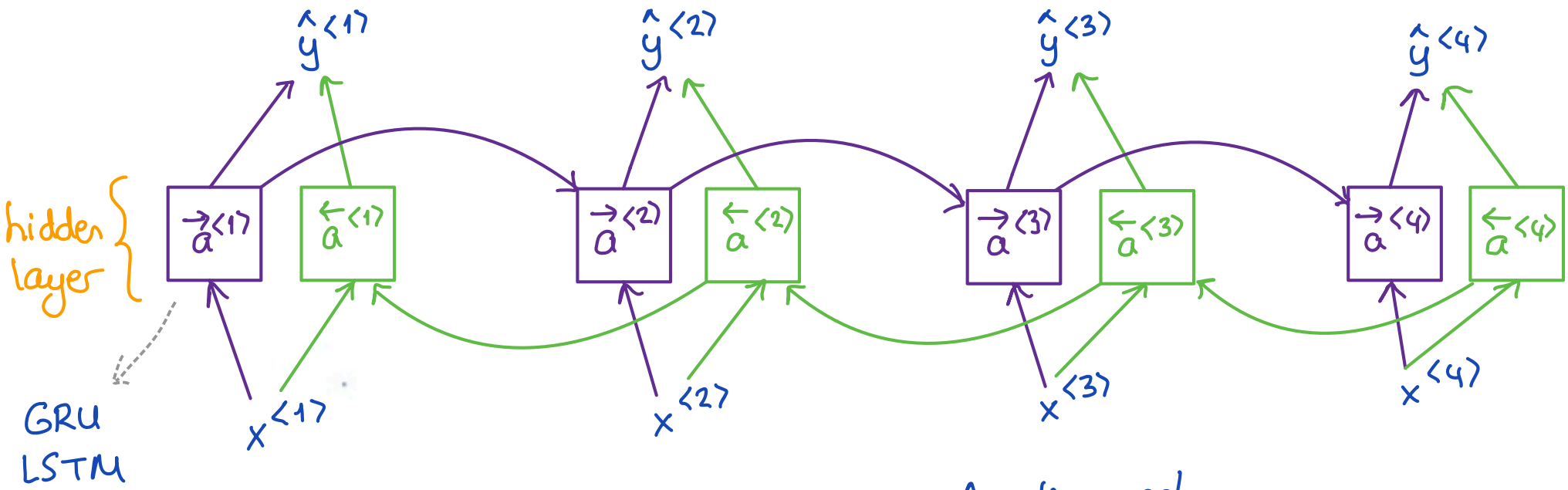
He said, “Teddy Roosevelt was a great President!”



- To tell if "Teddy" is a person's name, the first 3 words of the sentence are not sufficient.
- The above is a uni-directional sequential network, where each block is an RNN, GRU or LSTM unit.

Andrew Ng

Bidirectional RNN (BRNN)



$\rightarrow$: forward recurrent component

$\leftarrow$: backward recurrent component

$$\hat{y}^{<t>} = g(W_y[\vec{a}^{<t>}, \overleftarrow{a}^{<t>}] + b_y)$$

He said "Teddy Roosevelt ...."

Andrew Ng

- The <u>forward propagation</u> for the above network consists of two parts: forward recurrent computations (purple component) and backward recurrent computations.
  - After having computed $\vec{a}^{\langle 1\rangle}, \vec{a}^{\langle 2\rangle}, \vec{a}^{\langle 3\rangle}, \vec{a}^{\langle 4\rangle}$ and $\overleftarrow{a}^{\langle 1\rangle}, \overleftarrow{a}^{\langle 2\rangle}, \overleftarrow{a}^{\langle 3\rangle}, \overleftarrow{a}^{\langle 4\rangle}$ we can compute the predictions $\hat{y}^{\langle 1\rangle}, \hat{y}^{\langle 2\rangle}, \hat{y}^{\langle 3\rangle}$ and $\hat{y}^{\langle 4\rangle}$:

$$\hat{y}^{\langle t\rangle} = g\left(W_y\left[\vec{a}^{\langle t\rangle}, \overleftarrow{a}^{\langle t\rangle}\right] + b_y\right)$$

  - To compute $\hat{y}^{\langle 3\rangle}$ we need:
    - Forward computations using $x^{\langle 1\rangle}, x^{\langle 2\rangle}, x^{\langle 3\rangle}$, (i.e. $\vec{a}^{\langle 1\rangle}, \vec{a}^{\langle 2\rangle}, \vec{a}^{\langle 3\rangle}$)
    - Backward " " $x^{\langle 4\rangle}$ ($\overleftarrow{a}^{\langle 4\rangle}$ and $\overleftarrow{a}^{\langle 3\rangle}$)

- For NLP problems, a bidirectional RNN (BRNN) with LSTM blocks would be a reasonable first thing to try.
- <u>Disadvantage of BRNN</u>: You need the entire sequence of data before you can make predictions anywhere.
  - → e.g. For speech recognition, you need to wait for the entire utterance to end before you can actually process it.
  - → For real-time speech recognition there are other models used.



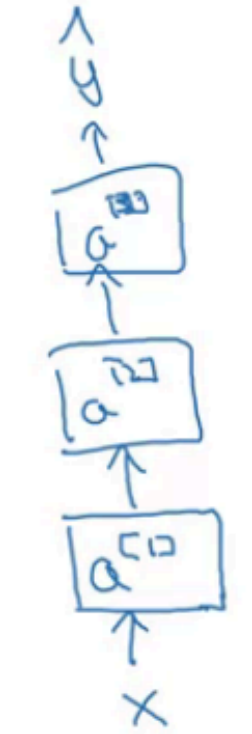
deeplearning.ai

Recurrent Neural Networks

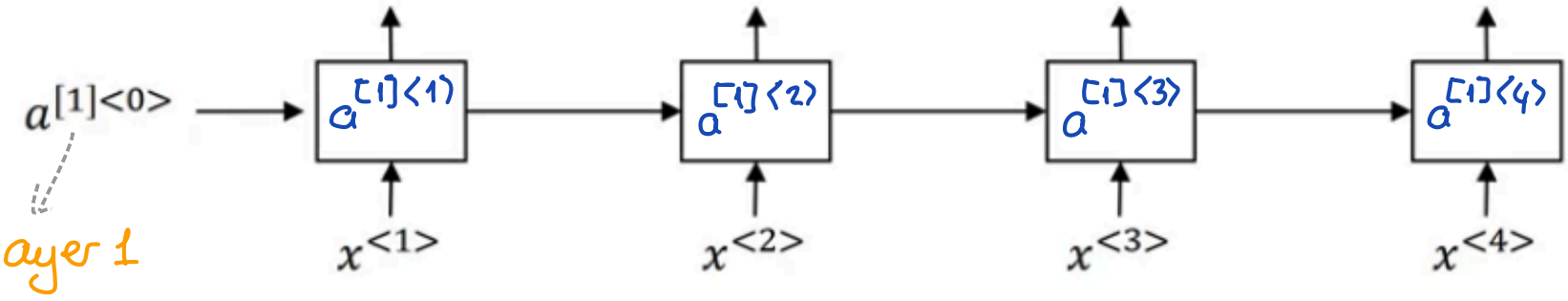
Deep RNNs

- Goal: Learn very complex functions by stacking multiple layers of RNNs to build deeper models.

Deep RNN example



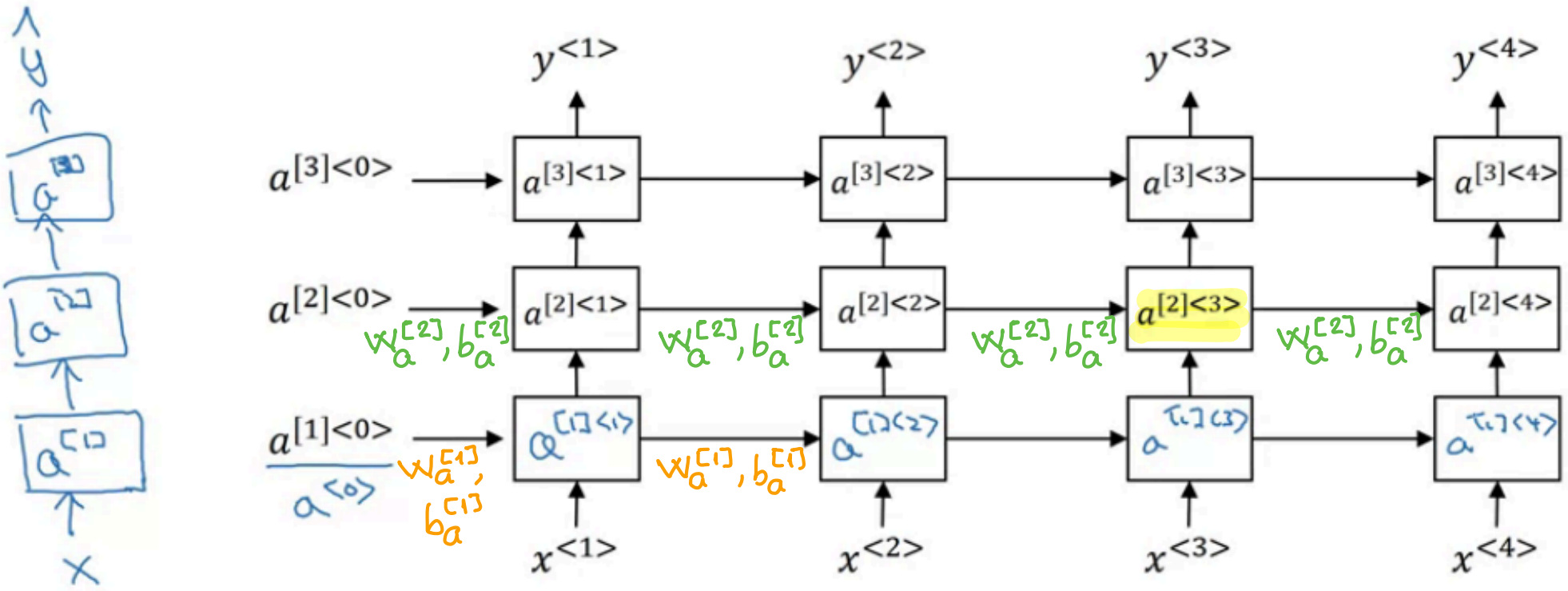
standard neural network



Andrew Ng

Deep RNN example

$a^{[l]<t>}$: layer l, time t.

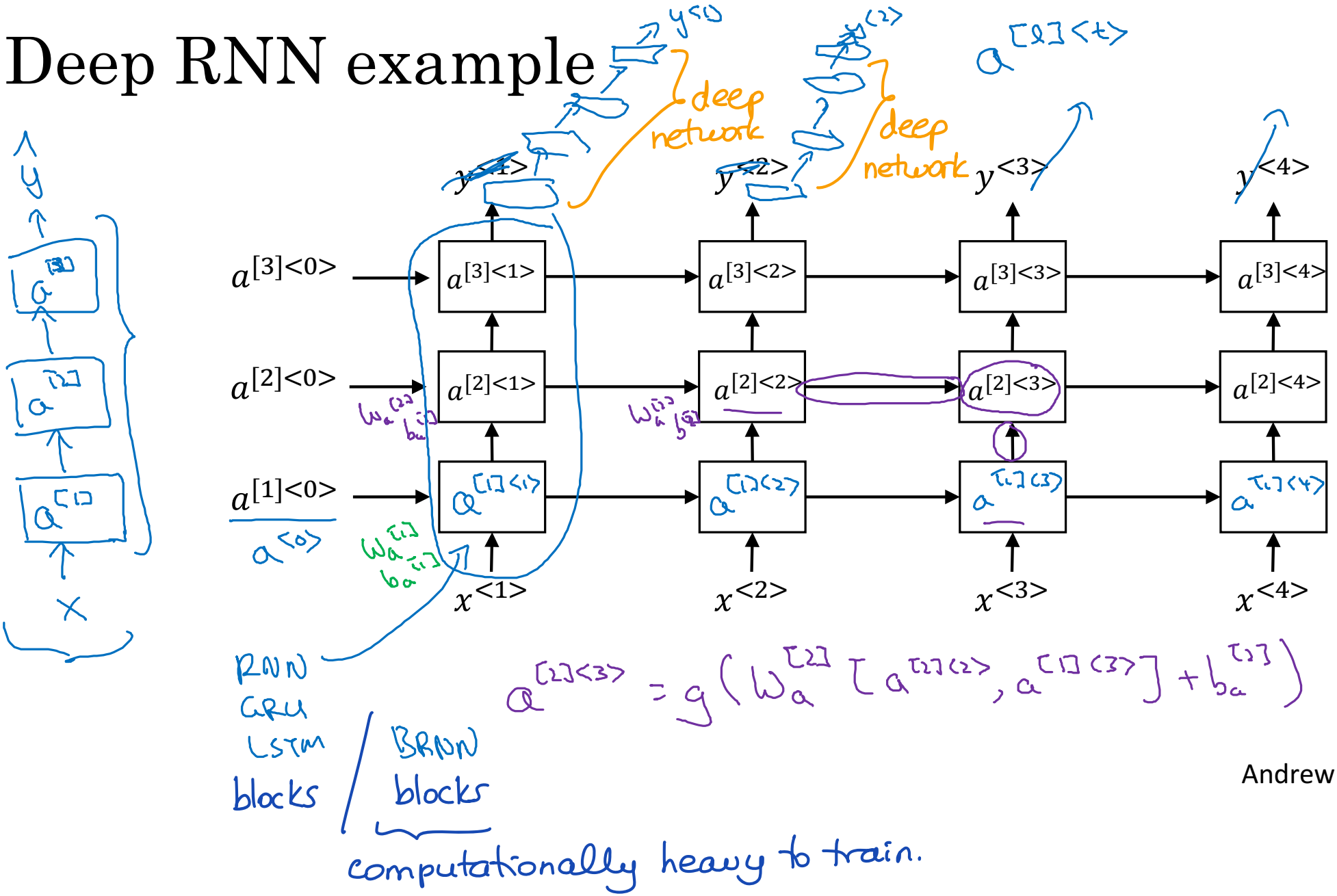


- A NN with 3 hidden layers.
- $a^{[2]<3>} = g(W_a^{[2]}[a^{[2]<2>}, a^{[1]<3>}] + b_a^{[2]})$

Andrew Ng

- for standard neural networks, we could have more than 100 layers.
- for RNNs, having 3 layers is already quite a lot, because of the temporal dimension.

Deep RNN example



Andrew Ng